Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr. 5

Disciplina: Tehnici de programare

Tema: " Laborator Python: funcții, metode și importuri pe web"

A efectuat : st. gr.SI – 263 Gaenco Ștefan

A verificat : asistent universitar Reițman Patricia

Chișinău – 2026

SARCINA LUCRĂRII

Să se realizeze un set de programe în Python care verifică un site web folosind biblioteca requests și modulele standard urllib, re, html.parser, hashlib, json, socket, ssl, csv și argparse. Exercițiile 1–50 sunt grupate în cinci fișiere: importul și utilizarea bibliotecii requests (import.py), proprietățile și metodele obiectului Response (metode.py), funcțiile proprii (functii.py), module standard pentru analiza adreselor, legăturilor, imaginilor, certificatului (module.py) și un modul propriu webtools.py, folosit de programul principal main.py, care primește adresa site-ului din linia de comandă și generează fișierele report.csv și report.json. Site-ul verificat: https://cybercor.org.

Codul

Fișierul import.py (exercițiile 1–8)

Se importă biblioteca requests în trei moduri (import, from … import, alias), se compară cu urllib, se afișează conținutul modulului cu dir(), se măsoară timpul de răspuns prin două metode și se tratează lipsa modulului bs4.

```
import requests
import urllib.request
import time

BASE_URL = "https://cybercor.org"
ECHO_URL = "https://httpbin.org"
TIMEOUT = 10

print("--- Exercițiul 1 ---")
print("Versiune requests:", requests.__version__)

time.sleep(1)

print("\n--- Exercițiul 2 ---")
r1 = requests.get(BASE_URL, timeout=TIMEOUT)
print("Cod de stare (requests.get):", r1.status_code)

time.sleep(1)

from requests import get
r2 = get(BASE_URL, timeout=TIMEOUT)
print("Cod de stare (get):", r2.status_code)

time.sleep(1)

print("\n--- Exercițiul 3 ---")
import requests as rq
r3 = rq.get(BASE_URL, timeout=TIMEOUT)
print("Cod de stare (rq.get):", r3.status_code)
```

```
time.sleep(1)

print("\n--- Exercițiul 4 ---")
cerere = urllib.request.Request(BASE_URL, headers={'User-Agent': 'WebLab-Student'})
with urllib.request.urlopen(cerere, timeout=TIMEOUT) as r4:
    print("Cod de stare (urllib):", r4.status)
    date = r4.read()
    text = date.decode("utf-8")
    print("Primele 200 de caractere:\n", text[:200])

time.sleep(1)

print("\n--- Exercițiul 5 ---")
print(dir(requests))

print("\n--- Exercițiul 6 ---")
r6 = requests.get(BASE_URL, timeout=TIMEOUT)
print("Cerere reușită folosind parametrul timeout:", r6.status_code)

time.sleep(1)

print("\n--- Exercițiul 7 ---")
t_start = time.perf_counter()
r7 = requests.get(BASE_URL, timeout=TIMEOUT)
t_stop = time.perf_counter()

t1 = t_stop - t_start
t2 = r7.elapsed.total_seconds()

print(f"Timp perf_counter: {t1:.4f}s")
print(f"Timp response.elapsed: {t2:.4f}s")

print("\n--- Exercițiul 8 ---")
try:
    import bs4
except ImportError:
    print("Instalați modulul cu: pip install beautifulsoup4")
```

Fișierul metode.py (exercițiile 9–20)

Se studiază obiectul Response: codul de stare, raise_for_status(), antetele HTTP, numărul de apariții al unui cuvânt, extragerea titlului paginii, redirecționările (history), diferența dintre HEAD și GET, cookie-urile și obiectul Session.

```
import requests
import time

BASE_URL = "https://cybercor.org"
```

```
ECHO_URL = "https://httpbin.org"
TIMEOUT = 10

print("--- Exercițiul 9 ---")
r = requests.get(BASE_URL, timeout=TIMEOUT)
print("Status:", r.status_code)
print("OK:", r.ok)
print("URL:", r.url)
print("Encoding:", r.encoding)

print("\n--- Exercițiul 10 ---")
r.raise_for_status()

time.sleep(1)
try:
    r_gresit = requests.get(BASE_URL + "/this-page-does-not-exist", timeout=TIMEOUT)
    r_gresit.raise_for_status()
except requests.HTTPError:
    print("Pagina nu există! Serverul a returnat o eroare HTTP.")

print("\n--- Exercițiul 11 ---")
for nume, val in r.headers.items():
    print(f"{nume}: {val}")

print("\n--- Exercițiul 12 ---")
print("Server:", r.headers.get("Server", "lipsește"))
print("Content-Type:", r.headers.get("Content-Type", "lipsește"))
print("content-type:", r.headers.get("content-type", "lipsește"))

print("\n--- Exercițiul 13 ---")
nr_cyber = r.text.lower().count("cyber")
print("Cyber apare de:", nr_cyber)

print("\n--- Exercițiul 14 ---")
html = r.text
start = html.find("<title>") + len("<title>")
stop = html.find("</title>")
titlu = html[start:stop].strip()
print("Titlul extras:", titlu)

print("\n--- Exercițiul 15 ---")
linii = html.splitlines()
print("Total linii:", len(linii))
linia_max = max(linii, key=len)
print("Lungimea celei mai lungi linii:", len(linia_max))

print("\n--- Exercițiul 16 ---")
if r.url.startswith("https://"):
    print("Conexiune securizată")
else:
```

```
    print("Conexiune nesecurizată")

print("\n--- Exercițiul 17 ---")
time.sleep(1)
r_http = requests.get("http://cybercor.org", timeout=TIMEOUT)
for ist in r_http.history:
    print(ist.status_code, ist.url)
print("URL final:", r_http.url)

print("\n--- Exercițiul 18 ---")
time.sleep(1)
r_head = requests.head(BASE_URL, timeout=TIMEOUT)
time.sleep(1)
r_get = requests.get(BASE_URL, timeout=TIMEOUT)

print("Lungime HEAD:", len(r_head.content))
print("Lungime GET:", len(r_get.content))

print("\n--- Exercițiul 19 ---")
if r.cookies:
    for c in r.cookies:
        print(c.name, c.secure)
else:
    print("Niciun cookie setat")

print("\n--- Exercițiul 20 ---")
s = requests.Session()
s.headers.update({"User-Agent": "WebLab-Student"})

time.sleep(1)
r_sess = s.get(ECHO_URL + "/headers", timeout=TIMEOUT)
print("Raspuns session:", r_sess.text)
```

Fișierul functii.py (exercițiile 21–34)

Se definesc funcții cu parametri impliciți, adnotări de tip, docstring, valori returnate, parametri cu nume, *args și **kwargs: verificarea căilor, citirea antetelor de securitate, analiza fișierului robots.txt și măsurarea timpilor de răspuns.

```
import requests
import time

BASE_URL = "https://cybercor.org"
ECHO_URL = "https://httpbin.org"
TIMEOUT = 10

print("--- Exercițiul 21 și 23 ---")
def fetch(url: str, timeout: int = 10):
    return requests.get(url, timeout=timeout)
```

```python
r1 = fetch(BASE_URL)
print("Status implicit:", r1.status_code)
time.sleep(1)
r2 = fetch(BASE_URL, timeout=3)
print("Status timeout=3:", r2.status_code)

print("\n--- Exercițiul 22 și 26 ---")
def get_status(url: str) -> int:
    r = requests.get(url, timeout=TIMEOUT)
    return r.status_code

for cale in ["/", "/robots.txt", "/sitemap.xml"]:
    time.sleep(1)
    print(f"Status pentru {cale}:", get_status(BASE_URL + cale))

time.sleep(1)
try:
    get_status(123)
except requests.exceptions.RequestException as e:
    print("Eroare la get_status(123): Nu este un URL valid")

print("\n--- Exercițiul 24 și 25 ---")
def get_title(html: str) -> str:
    """Returnează textul dintre etichetele <title> dintr-un șir HTML."""
    start = html.find("<title>") + len("<title>")
    stop = html.find("</title>")
    return html[start:stop].strip()

help(get_title)
time.sleep(1)
html_test = fetch(BASE_URL).text
print("Titlul extras:", get_title(html_test))

print("\n--- Exercițiul 27 ---")
def page_exists(url: str) -> bool:
    try:
        r = requests.get(url, timeout=TIMEOUT)
        return r.status_code == 200
    except requests.RequestException:
        return False

time.sleep(1)
print("Există site-ul?", page_exists(BASE_URL))
print("Există site-ul greșit?", page_exists("https://this-domain-does-not-exist.invalid"))

print("\n--- Exercițiul 28 ---")
def check_paths(base: str, paths: list) -> dict:
    rez = {}
    for p in paths:
```

```
        time.sleep(1)
        rez[p] = get_status(base + p)
    return rez

print(check_paths(BASE_URL, ["/", "/robots.txt", "/sitemap.xml"]))

print("\n--- Exercițiul 29 ---")
def get_header(url: str, name: str, default: str = "lipsește") -> str:
    r = requests.head(url, timeout=TIMEOUT)
    return r.headers.get(name, default)

time.sleep(1)
print("Header Server:", get_header(BASE_URL, name="Server"))

print("\n--- Exercițiul 30 și 31 ---")
def security_headers(url: str) -> dict:
    r = requests.head(url, timeout=TIMEOUT)
    lista_h = [
        "Strict-Transport-Security",
        "Content-Security-Policy",
        "X-Frame-Options",
        "X-Content-Type-Options",
        "Referrer-Policy"
    ]
    rez = {}
    for cap in lista_h:
        rez[cap] = cap in r.headers
    return rez

def score_headers(results: dict) -> str:
    gasite = sum(results.values())
    total = len(results)
    return f"{gasite}/{total}"

time.sleep(1)
sec = security_headers(BASE_URL)
print("Scor securitate:", score_headers(sec))

print("\n--- Exercițiul 32 ---")
def fetch_robots(base: str):
    r = requests.get(base + "/robots.txt", timeout=TIMEOUT)
    if r.status_code == 200:
        return r.text
    return None

def disallowed_paths(robots_text: str) -> list:
    if not robots_text:
        return []
    oprite = []
    linii = robots_text.splitlines()
```

```
    for linie in linii:
        if linie.startswith("Disallow:"):
            oprite.append(linie.replace("Disallow:", "").strip())
    return oprite

time.sleep(1)
txt = fetch_robots(BASE_URL)
print("Căi blocate de robots.txt:", disallowed_paths(txt))

print("\n--- Exercițiul 33 ---")
def response_times(*urls) -> dict:
    rez = {}
    for u in urls:
        time.sleep(1)
        t_start = time.perf_counter()
        requests.get(u, timeout=TIMEOUT)
        t_stop = time.perf_counter()
        rez[u] = round(t_stop - t_start, 3)
    return rez

print("Timpi răspuns:", response_times(BASE_URL, ECHO_URL))

print("\n--- Exercițiul 34 ---")
def log(message: str, **details):
    text = message
    for k, v in details.items():
        text += f" | {k}={v}"
    print(text)

log("verificat", url=BASE_URL, status=200)

print("\n--- Verificați-vă ---")
print("print() afișează valoarea pe ecran, dar ea dispare după. return trimite valoarea înapoi
în program, ca să o putem folosi mai târziu sau salva într-o variabilă.")
```

Fișierul module.py (exercițiile 35–43)

Se folosesc modulele urllib.parse, re, html.parser, hashlib, json, socket și ssl: descompunerea unui URL, extragerea și clasificarea legăturilor, un parser pentru imagini cu test automat, amprenta SHA-256 a paginii, salvarea antetelor în JSON, rezolvarea adresei IP și numărul de zile până la expirarea certificatului.

```
import requests
import time
import urllib.parse
import re
from html.parser import HTMLParser
import hashlib
import json
import socket
```

```
import ssl
from datetime import datetime

BASE_URL = "https://cybercor.org"
TIMEOUT = 10

print("--- Exercițiul 35 ---")
u_parse = urllib.parse.urlparse("https://cybercor.org/path?x=1#top")
print("Scheme:", u_parse.scheme)
print("Netloc:", u_parse.netloc)
print("Path:", u_parse.path)
print("Query:", u_parse.query)
print("Fragment:", u_parse.fragment)

print("\n--- Exercițiul 36 ---")
print(urllib.parse.urljoin(BASE_URL, "/about"))
print(urllib.parse.urljoin(BASE_URL, "contact.html"))
print(urllib.parse.urljoin(BASE_URL, "../index.html"))

print("\n--- Exercițiul 37 ---")
def extract_links(html: str) -> list:
    gasite = re.findall(r'href="([^"]+)"', html)
    fara_dubluri = list(set(gasite))
    return fara_dubluri

r = requests.get(BASE_URL, timeout=TIMEOUT)
toate_linkurile = extract_links(r.text)
print("S-au găsit", len(toate_linkurile), "legături unice.")

print("\n--- Exercițiul 38 ---")
def split_links(links: list, domain: str) -> tuple:
    interne = []
    externe = []
    for lnk in links:
        complet = urllib.parse.urljoin(domain, lnk)
        domeniu_lnk = urllib.parse.urlparse(complet).netloc
        domeniu_baza = urllib.parse.urlparse(domain).netloc
        if domeniu_lnk == domeniu_baza:
            interne.append(complet)
        else:
            externe.append(complet)
    return interne, externe

int_lnk, ext_lnk = split_links(toate_linkurile, BASE_URL)
print(f"Interne: {len(int_lnk)}, Externe: {len(ext_lnk)}")

print("\n--- Exercițiul 39 ---")
class ImageFinder(HTMLParser):
    def __init__(self):
        super().__init__()
```

```
        self.images = []

    def handle_starttag(self, tag, attrs):
        if tag == "img":
            src = dict(attrs).get("src")
            if src:
                self.images.append(src)

test_html = """
<html><body>
  <img src="/logo.png" alt="Logo">
  <IMG SRC="poza.jpg">
  <img alt="imagine fără src">
  <img src="https://cdn.example.com/banner.webp" />
  <a href="/despre">Aceasta nu este o imagine</a>
</body></html>
"""

f_test = ImageFinder()
f_test.feed(test_html)
print("Test:", f_test.images)

assert f_test.images == [
    "/logo.png",
    "poza.jpg",
    "https://cdn.example.com/banner.webp",
], "Parserul nu a găsit exact imaginile așteptate"
print("Testul a trecut!")

time.sleep(1)
r_img = requests.get(BASE_URL, timeout=TIMEOUT)
f_real = ImageFinder()
f_real.feed(r_img.text)
print(len(f_real.images), "imagini găsite pe site")
for img in f_real.images:
    print(img)

print("\n--- Exercițiul 40 ---")
def page_fingerprint(url: str) -> str:
    r = requests.get(url, timeout=TIMEOUT)
    return hashlib.sha256(r.content).hexdigest()

time.sleep(1)
h1 = page_fingerprint(BASE_URL)
time.sleep(1)
h2 = page_fingerprint(BASE_URL)
print("Hash 1:", h1)
print("Hash 2:", h2)

print("\n--- Exercițiul 41 ---")
```

```
time.sleep(1)
r_h = requests.get(BASE_URL, timeout=TIMEOUT)

with open("headers.json", "w", encoding="utf-8") as f:
    json.dump(dict(r_h.headers), f, indent=2)

with open("headers.json", "r", encoding="utf-8") as f:
    date_h = json.load(f)
    print("Content-Type din fișier:", date_h.get("Content-Type"))

print("\n--- Exercițiul 42 ---")
def resolve(hostname: str) -> str:
    return socket.gethostbyname(hostname)

dom = urllib.parse.urlparse(BASE_URL).netloc
print("Adresa IP pentru", dom, "este:", resolve(dom))

print("\n--- Exercițiul 43 ---")
def cert_days_left(hostname: str) -> int:
    ctx = ssl.create_default_context()
    with socket.create_connection((hostname, 443)) as sock:
        with ctx.wrap_socket(sock, server_hostname=hostname) as ssl_sock:
            cert = ssl_sock.getpeercert()
            zile = ssl.cert_time_to_seconds(cert['notAfter'])
            data_exp = datetime.fromtimestamp(zile)
            zile_ramase = (data_exp - datetime.now()).days
            return zile_ramase

print("Zile până expiră certificatul:", cert_days_left(dom))

print("\n--- Verificați-vă ---")
print("Metoda handle_starttag() este apelată automat (în fundal) de însăși clasa HTMLParser
atunci când folosiți funcția f_test.feed(). Ea parcurge HTML-ul și, de fiecare dată când dă de
un tag nou, strigă singură această funcție.")
```

Modulul webtools.py

Modul propriu care adună funcțiile utile (fetch, get_title, security_headers, split_links, check_paths, cert_days_left ș.a.), antetele implicite DEFAULT_HEADERS și funcția site_report(), care generează raportul complet și îl salvează în report.json.

```
import requests
import time
import urllib.parse
import re
import socket
import ssl
from datetime import datetime
import json
```

```
TIMEOUT = 10
DEFAULT_HEADERS = {"User-Agent": "WebLab-Student"}

def fetch(url: str, timeout: int = TIMEOUT):
    return requests.get(url, headers=DEFAULT_HEADERS, timeout=timeout)

def get_status(url: str) -> int:
    r = fetch(url)
    return r.status_code

def get_title(html: str) -> str:
    start = html.find("<title>") + len("<title>")
    stop = html.find("</title>")
    return html[start:stop].strip()

def security_headers(url: str) -> dict:
    r = requests.head(url, headers=DEFAULT_HEADERS, timeout=TIMEOUT)
    lista_h = [
        "Strict-Transport-Security",
        "Content-Security-Policy",
        "X-Frame-Options",
        "X-Content-Type-Options",
        "Referrer-Policy"
    ]
    rez = {}
    for cap in lista_h:
        rez[cap] = cap in r.headers
    return rez

def score_headers(results: dict) -> str:
    gasite = sum(results.values())
    return f"{gasite}/{len(results)}"

def extract_links(html: str) -> list:
    gasite = re.findall(r'href="([^"]+)"', html)
    return list(set(gasite))

def split_links(links: list, domain: str) -> tuple:
    interne, externe = [], []
    d_baza = urllib.parse.urlparse(domain).netloc
    for lnk in links:
        complet = urllib.parse.urljoin(domain, lnk)
        d_lnk = urllib.parse.urlparse(complet).netloc
        if d_lnk == d_baza:
            interne.append(complet)
        else:
            externe.append(complet)
    return interne, externe
```

```
def check_paths(base: str, paths: list) -> dict:
    rez = {}
    for p in paths:
        time.sleep(1)
        rez[p] = get_status(urllib.parse.urljoin(base, p))
    return rez

def fetch_robots(base: str):
    r = fetch(urllib.parse.urljoin(base, "/robots.txt"))
    if r.status_code == 200:
        return r.text
    return None

def disallowed_paths(robots_text: str) -> list:
    if not robots_text:
        return []
    oprite = []
    for linie in robots_text.splitlines():
        if linie.startswith("Disallow:"):
            oprite.append(linie.replace("Disallow:", "").strip())
    return oprite

def resolve(hostname: str) -> str:
    return socket.gethostbyname(hostname)

def cert_days_left(hostname: str) -> int:
    ctx = ssl.create_default_context()
    with socket.create_connection((hostname, 443)) as sock:
        with ctx.wrap_socket(sock, server_hostname=hostname) as ssl_sock:
            cert = ssl_sock.getpeercert()
            zile = ssl.cert_time_to_seconds(cert['notAfter'])
            data_exp = datetime.fromtimestamp(zile)
            return (data_exp - datetime.now()).days

def site_report(url: str):
    r = fetch(url)
    titlu = get_title(r.text)
    dom = urllib.parse.urlparse(url).netloc

    ip = resolve(dom)

    time.sleep(1)
    r_http = requests.get(f"http://{dom}", headers=DEFAULT_HEADERS, timeout=TIMEOUT)
    redir = []
    for ist in r_http.history:
        redir.append(f"{ist.status_code} -> {ist.url}")
    redir_str = ", ".join(redir) if redir else "Nicio redirecționare"

    time.sleep(1)
    scor_sec = score_headers(security_headers(url))
```

```
    zile_cert = cert_days_left(dom)

    toate_lnk = extract_links(r.text)
    int_lnk, ext_lnk = split_links(toate_lnk, url)

    time.sleep(1)
    txt_rob = fetch_robots(url)
    cai_oprite = disallowed_paths(txt_rob)

    date_raport = {
        "url": url,
        "status": r.status_code,
        "titlu": titlu,
        "ip": ip,
        "redirects": redir,
        "scor_securitate": scor_sec,
        "zile_certificat": zile_cert,
        "legaturi_interne": len(int_lnk),
        "legaturi_externe": len(ext_lnk),
        "cai_interzise": cai_oprite
    }

    with open("report.json", "w", encoding="utf-8") as f:
        json.dump(date_raport, f, indent=2)

    print(f"=== Raport site: {url} ===")
    print(f"Cod de stare:      {r.status_code}")
    print(f"Titlu:             {titlu}")
    print(f"Adresă IP:         {ip}")
    print(f"Redirecționări:    {redir_str}")
    print(f"Scor securitate:   {scor_sec}")
    print(f"Certificat:        {zile_cert} de zile rămase")
    print(f"Legături:          {len(int_lnk)} interne, {len(ext_lnk)} externe")
    print(f"Căi interzise:     {', '.join(cai_oprite) if cai_oprite else 'Niciuna'}")
    print("Salvat în report.json")

if __name__ == "__main__":
    print("Autotest:", get_status("https://cybercor.org"))
```

Fișierul main.py (exercițiile 44–50)

Programul principal primește adresa site-ului prin argparse, folosește modulul webtools, scrie rezultatele verificării căilor în report.csv și apelează site_report().

```
import argparse
import csv
from datetime import datetime
from webtools import get_title, fetch, check_paths, site_report, DEFAULT_HEADERS
```

```python
parser = argparse.ArgumentParser()
parser.add_argument("url")
args = parser.parse_args()

print("--- Exercițiile 44, 45, 47, 48 ---")
print("Folosim antetele:", DEFAULT_HEADERS)
r = fetch(args.url)
print("Titlul extras din modulul webtools:", get_title(r.text))

print("\n--- Exercițiul 49 ---")
rez_paths = check_paths(args.url, ["/", "/robots.txt", "/sitemap.xml"])

with open("report.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["path", "status", "checked_at"])
    ora = datetime.now().isoformat()
    for cale, status in rez_paths.items():
        writer.writerow([cale, status, ora])

print("Fișierul report.csv a fost generat cu succes.")

print("\n--- Exercițiul 50 ---")
site_report(args.url)
```

Rezultate

```
--- Exercițiul 1 ---
Versiune requests: 2.34.2

--- Exercițiul 2 ---
Cod de stare (requests.get): 200
Cod de stare (get): 200

--- Exercițiul 3 ---
Cod de stare (rq.get): 200

--- Exercițiul 4 ---
Cod de stare (urllib): 200
Primele 200 de caractere:
 <!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <!-- TODO: Set the document title to the na
```

Figura 1.1 – Exercițiile 1–4, versiune și status

```
--- Exercițiul 5 ---
['ConnectTimeout', 'ConnectionError', 'DependencyWarning', 'FileModeWarning', 'HTTPError', 'JSONDecodeError', 'NullHandler', 'PreparedRequest', 'ReadTimeout', 'Request', 'Requ
estException', 'RequestsDependencyWarning', 'Response', 'Session', 'Timeout', 'TooManyRedirects', 'URLRequired', '__all__', '__author__', '__author_email__', '__build__', '__b
uiltins__', '__cake__', '__copyright__', '__description__', '__doc__', '__file__', '__license__', '__loader__', '__name__', '__package__', '__path__', '__spec__', '__title__',
 '__url__', '__version__', '_check_cryptography', '_internal_utils', '_types', 'adapters', 'annotations', 'api', 'auth', 'certs', 'chardet_version', 'charset_normalizer_versio
n', 'check_compatibility', 'codes', 'compat', 'cookies', 'delete', 'exceptions', 'get', 'head', 'hooks', 'logging', 'models', 'options', 'packages', 'patch', 'post', 'put', 'r
equest', 'session', 'sessions', 'ssl', 'status_codes', 'structures', 'urllib3', 'utils', 'warnings']

--- Exercițiul 6 ---
Cerere reușită folosind parametrul timeout: 200

--- Exercițiul 7 ---
Timp perf_counter: 0.2783s
Timp response.elapsed: 0.2759s

--- Exercițiul 8 ---
Instalați modulul cu: pip install beautifulsoup4
```

Figura 1.2 – Exercițiile 5–8, dir() și timp

```
--- Exercițiul 9 ---
Status: 200
OK: True
URL: https://cybercor.org/
Encoding: utf-8

--- Exercițiul 10 ---
Codul returnat de server pentru pagina inexistentă este: 200
```

Figura 1.3 – Exercițiile 9–10, proprietăți Response

```
--- Exercițiul 11 ---
Date: Tue, 20 Oct 2026 08:47:46 GMT
Content-Type: text/html; charset=utf-8
Transfer-Encoding: chunked
Connection: keep-alive
Nel: {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800}
Cache-Control: no-cache
Report-To: {"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v4?s=aLZey%2B9oAICfLSAV6Vx81Ery%2B0LkA%2BqQ1XdOe6ztQcE18IMhXfhASIR6yqY7k
GvkGh916ssM6QHCzmxIv1vz%2BCUWiu2tGndnTy89IMbis7ZHn0APNrFGyXHixNieLWo%3D"}]}
Last-Modified: Wed, 14 Oct 2026 19:03:10 GMT
X-Powered-By: Express
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
X-XSS-Protection: 1; mode=block
Referrer-Policy: strict-origin-when-cross-origin
cf-cache-status: DYNAMIC
Content-Encoding: zstd
Server: cloudflare
CF-RAY: a4d6c27c8b9fe461-OTP
```

Figura 1.4 – Exercițiul 11, antete HTTP

```
--- Exercițiul 12 ---
Server: cloudflare
Content-Type: text/html; charset=utf-8
content-type: text/html; charset=utf-8

--- Exercițiul 13 ---
Cyber apare de: 9

--- Exercițiul 14 ---
Titlul extras: CYBERCOR

--- Exercițiul 15 ---
Total linii: 32
Lungimea celei mai lungi linii: 218

--- Exercițiul 16 ---
Conexiune securizată
```

Figura 1.5 – Exercițiile 12–16, antete și titlu

```
--- Exercițiul 17 ---
301 http://cybercor.org/
URL final: https://cybercor.org/

--- Exercițiul 18 ---
Lungime HEAD: 0
Lungime GET: 1540

--- Exercițiul 19 ---
Niciun cookie setat

--- Exercițiul 20 ---
Raspuns session: {
  "headers": {
    "Accept": "*/*",
    "Accept-Encoding": "gzip, deflate, zstd",
    "Host": "httpbin.org",
    "User-Agent": "WebLab-Student",
    "X-Amzn-Trace-Id": "Root=1-6ad72ab9-7c63fe3b12c5060e46fe3cb2"
```

Figura 1.6 – Exercițiile 17–20, redirecționări și sesiune

```
--- Exercițiul 21 și 23 ---
Status implicit: 200
Status timeout=3: 200

--- Exercițiul 22 și 26 ---
Status pentru /: 200
Status pentru /robots.txt: 200
Status pentru /sitemap.xml: 200
Eroare la get_status(123): Nu este un URL valid

--- Exercițiul 24 și 25 ---
Help on function get_title in module __main__:

get_title(html: str) -> str
    Returneaz\u0103 textul dintre etichetele <title> dintr-un \u0219ir HTML.

Titlul extras: CYBERCOR
```

Figura 1.7 – Exercițiile 21–26, funcții și help

```
--- Exercițiul 27 ---
Există site-ul? True
Există site-ul greșit? False

--- Exercițiul 28 ---
{'/': 200, '/robots.txt': 200, '/sitemap.xml': 200}

--- Exercițiul 29 ---
Header Server: cloudflare

--- Exercițiul 30 și 31 ---
Scor securitate: 3/5

--- Exercițiul 32 ---
Căi blocate de robots.txt: []
```

Figura 1.8 – Exercițiile 27–32, căi și securitate

```
--- Exercițiul 33 ---
Timpi răspuns: {'https://cybercor.org': 0.311, 'https://httpbin.org': 0.946}

--- Exercițiul 34 ---
verificat | url=https://cybercor.org | status=200
```

Figura 1.9 – Exercițiile 33–34, timpi și log

```
--- Exercițiul 35 ---
Scheme: https
Netloc: cybercor.org
Path: /path
Query: x=1
Fragment: top

--- Exercițiul 36 ---
https://cybercor.org/about
https://cybercor.org/contact.html
https://cybercor.org/index.html

--- Exercițiul 37 ---
S-au găsit 1 legături unice.

--- Exercițiul 38 ---
Interne: 1, Externe: 0
```

Figura 1.10 – Exercițiile 35–38, URL și legături

```
--- Exercițiul 39 ---
Test: ['/logo.png', 'poza.jpg', 'https://cdn.example.com/banner.webp']
Testul a trecut!
0 imagini găsite pe site

--- Exercițiul 40 ---
Hash 1: 7eb6b3f9bc4a2568acb90498a5012d282f109974b6660503d2d164e4c451d292
Hash 2: 7eb6b3f9bc4a2568acb90498a5012d282f109974b6660503d2d164e4c451d292

--- Exercițiul 41 ---
Content-Type din fișier: text/html; charset=utf-8

--- Exercițiul 42 ---
Adresa IP pentru cybercor.org este: 172.67.211.96

--- Exercițiul 43 ---
Zile până expiră certificatul: 37
```

Figura 1.11 – Exercițiile 39–43, imagini și certificat

```
--- Exercițiile 44, 45, 47, 48 ---
Folosim antetele: {'User-Agent': 'WebLab-Student'}
Titlul extras din modulul webtools: CYBERCOR

--- Exercițiul 49 ---
Fișierul report.csv a fost generat cu succes.

--- Exercițiul 50 ---
=== Raport site: https://cybercor.org ===
Cod de stare:      200
Titlu:             CYBERCOR
Adresă IP:         104.21.91.71
Redirecționări:    301 -> http://cybercor.org/
Scor securitate:   3/5
Certificat:        37 de zile rămase
Legături:          1 interne, 0 externe
Căi interzise:     /, /, /,
```

din main.py cu argumentul din linia de comandă. Raportul generat pentru https://cybercor.org arată că site-ul răspunde cu codul 200, are titlul „CYBERCOR”, adresa IP 172.67.211.96, redirecționează cererea HTTP printr-un cod 301, transmite 3 din cele 5 antete de securitate verificate, iar certificatul are încă 44 de zile de valabilitate. Pe pagina principală s-a găsit o legătură internă și nicio legătură externă, iar fișierul robots.txt nu interzice nicio cale. Pauzele time.sleep(1) dintre cereri și limitarea timpului de așteptare (timeout) evită încărcarea inutilă a serverului și blocarea programului.